AI 서비스 MINI-PROJECT · 과제 1번

Base AI Agent의 RunBook 선택 개선

RAG 기반 RunBook 선택 · 데이터 확장 · 평가 하네스 구축

광주 3반 79번 박건우 · 2026-10-22

FastAPI + LangGraph v2 + MCP

all_total_vue_student

Git: github.com/pigpgw/skala-ai-service-mini

문제

장애 유형을 고르는 부분이 틀리고 있었다

판정 로직이 한 함수에 압축돼 있었고, 규칙 순서가 곧 결과였습니다.

```
# agent/graph.py::_classify_incident_type()
latency → pool → cpu → memory → timeout
  → network → disk → down
# 순서대로 보다가 "처음 맞는 것"을 선택
```

- **규칙 순서가 증상 우선순위와 무관**
  CPU가 timeout보다 앞에 있어, 타임아웃 질의가 `cpu-spike`가 됨
- **질의문을 아예 보지 않음**
  "디스크 부족" 신고가 지연 알림에 가려 `high-latency`가 됨
- **매칭 실패 시 무조건 `db-down`**
  "아무 장애나 db-down 대응을 내놔라"

근거

먼저 고쳐 볼 자리를 찾았다

고치기 전에, 같은 입력으로 뭐가 틀리는지 고정해서 재기로 했습니다.

8/12

keyword 기준선 정확도 = 0.6667

4개

오분류 (B07·B09·B10·B11)

0.6667

eval/baseline_keyword.json

- **B07** — gateway(타임아웃) + payments(CPU)를 함께 조회하면 CPU가 먼저 잡힘
- **B09** — memory 규칙이 disk 규칙보다 앞에 있음
- **B10·B11** — 질의 텍스트를 읽지 않아서 생긴 오분류

개선 1

RunBook 선택을 키워드 규칙 → 임베딩 의미 검색으로

확신하지 못하면 예전 동작으로 돌아가는 게이트를 두었습니다.

판정 5분기 — 실패하면 keyword 폴백

- disabled / index_unavailable
- embed_failed
- low_score / low_margin
- **그 외 → RAG 판정**

입력: [사용자 질문][시스템][상태][알림]

- `agent/rag/embedder.py` — Ollama qwen3-embedding:0.6b (1024 차원)
- `agent/rag/index.py` — 10문서 인덱스 + JSON 캐시, `index_sha256` 기록
- `agent/rag/classifier.py` — 5분기 판정 + 폴백
- `agent/graph.py::_select_incident_type()` → `replan_check_node`
- 근거 노출: `/health.rag`, `/chat` metadata, SSE `ReplanCheck`

개선 2

데이터 확장 — 시스템 6→12, 런북 6→10

파일 선택을 한 곳으로 모았습니다. 다섯 곳에 흩어져 있었습니다.

- 신규 시스템 6개 — search·notify·session·media·inventory·report-service
- 신규 런북 4개 — **queue-backlog**, **cert-expiry**, **connection-pool**, **replication-lag**
- `DATA_PROFILE=base|extended` — 결정 지점은 `agent/data_profile.py` 한 곳
- 기존 `monitoring.json`은 덮어쓰지 않고 기준선으로 보존
- `billing-service`는 일부러 제외 — 미지 시스템 처리 회귀 검증용

왜 한 곳으로 모았나

경로가 다섯 곳에 흩어져 있으면 시스템만 늘리고 나머지가 6개 목록으로 잘리는 **가짜 확장**이 됩니다. 측정이 조용히 틀어지는 경우가 가장 위험하다고 판단했습니다.

개선 3

평가 하네스 — 실행 1회 = 폴더 1개

Before/After가 같은 조건인지 확인되려면, 그 조건이 파일로 남아야 했습니다.

- `eval/run_eval.py` — 실행 1회 = `artifacts/runs/<run_id>/`
  manifest.json(조건·해시·git) · metrics.json · requests/responses jsonl
- `eval/metrics.py` — accuracy · macro-F1 · 혼동행렬 · 지연 percentile
- `eval/labels_extended.json` — 40문항(10유형 × 4카테고리), **측정 전 동결**
- `tests/` — unit 5종 + integration 2종 = **50개 통과**

실행 1회의 구조

```
artifacts/runs/20261020-152756_fake_after-rag/
  manifest.json   // 조건·해시·git·env
  metrics.json    // 지표·혼동행렬·오분류
  requests.jsonl  // 입력 1건 = 1줄
  responses.jsonl // 출력 1건 = 1줄
  notes.md        // 관찰 기록
```

결과

Before / After — 같은 조건, 같은 입력

fake 백엔드 · DATA_PROFILE=extended · qwen3-embedding:0.6b · 임계값 0.50 / 0.03

지표	Before (keyword)	After (RAG)	차이
40문항 정확도	13/40 = 0.3250	34/40 = 0.8500	+0.5250
macro-F1	0.2836	0.8521	+0.5685
E2E 5문항 (실서버)	1/5 = 0.2000	5/5 = 1.0000	+0.8000
분류 지연 p50	0.0ms	32.43ms	+32.43ms
E2E 요청 지연 p50	750.76ms	819.73ms	+68.97ms
비용	$0.000000	$0.000000	0 (로컬)

신규 4유형 F1 — queue-backlog 0.000→**1.000**, cert-expiry 0.000→**1.000**, connection-pool 0.000→**1.000**, replication-lag 0.000→**0.857**

해석

좋았던 것과, 안 좋았던 것

데이터 확장은 정확도를 올리지 못했다

base 34/40 ≡ extended 34/40 — 문항 단위로 동일. 라벨이 질의에 증상을 적어 둬서 RAG가 질의만 읽어도 맞고, 시스템 알림은 중복 신호가 됐습니다. **그래서 "확장으로 정확도 상승"이라고 쓰지 않았습니다.**

대신 조사 커버리지(unknown) 18.75% → 0%

margin 게이트는 2문항을 아꼈다

임계값을 0.00으로 풀면 36/40이 되지만, 그 2문항은 1·2위 유사도 차이가 0.0155·0.0000인 동점입니다. 1단계에서 정답이 뒤집힌 사례도 이런 동점이었습니다.

0.03 유지 + 비용 2문항 기록

RAG의 비용은 임베딩 1회 호출입니다. 분류 1건당 +32.43ms, 실서버 요청 전체로는 +68.97ms — 요청 시간의 약 9% 수준이었습니다.

한계

아직 확인하지 않은 것

- **40문항 라벨을 제가 만들었다** — 자기 평가라서 카테고리별로 나눠서 보고했습니다. 전체 수치로 일반화하지 않습니다
- **keyword 폴백 규칙은 그대로** — 신규 유형마다 함께 갱신해야 합니다
- **비용 0은 로컬 백엔드 때문** — 실 API 비용은 별도 측정 필요

다음에 확인할 것

- 실제 장애 티켓·운영 로그로 라벨 세트 재구축
- 동시 요청 부하에서의 지연 측정
- 실 OpenAI 백엔드로 비용·지연 재측정
- keyword 규칙 우선순위를 데이터 기반으로 재설계할지 판단

시연 · 정리

판정을 바꾼 게 아니라,
판단의 근거를 남기고 측정할 수 있게 바꾼 것

라이브 데모 (3분)

- `curl -s localhost:8100/health` → rag 블록 · 12시스템
- 채팅: auth-service p95 2.4초 → **high-latency**
- 채팅: search-service 큐 1.2만 건 → **queue-backlog** (신규 유형)
- 채팅: billing-service 장애 → **unknown + 폴백**

그리면

RAG 도입으로 40문항 정확도 **0.3250** → **0.8500**, macro-F1 **0.2836** → **0.8521**.

대신 비용 **+32.43ms**, margin 게이트 비용 **2문항**, 데이터 확장의 정확도 효과 **0** — 이 한계를 수치 그대로 남겼습니다.